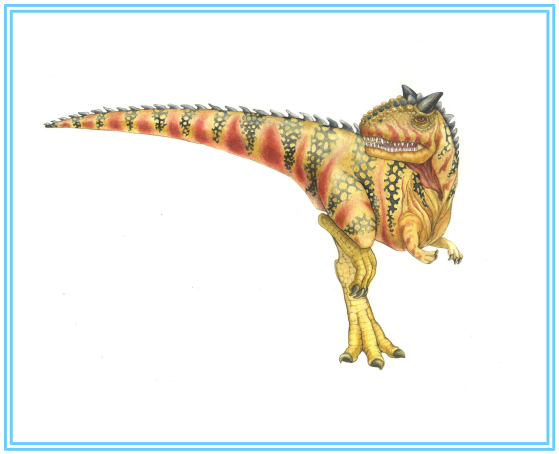
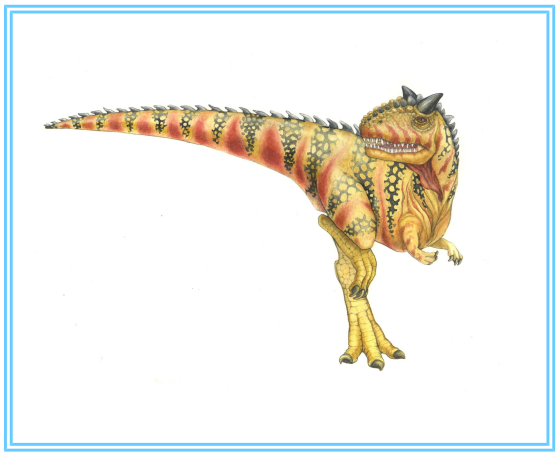
Review & Excercise



Silberschatz, Galvin and Gagne ©2009

第1章 导论



Silberschatz, Galvin and Gagne ©2009



第1章：导论 概要总结

- 操作系统概念：
  - 特征
  - 功能
  - 种类
- 用户态和内核态（目态和管态）





1. 操作系统的定义及特征

操作系统是管理和控制计算机系统中软硬件资源，合理组织计算机工作流程，方便用户操作使用机器的程序的集合。

基本特征:

- 执行的并发性
- 资源的共享性
- 操作的异步性





2. 操作系统的功能

processor

(1). 处理机管理(包括: 进程管理)

(2). 存储管理

(3). 设备管理

(4). 文件系统管理

(5). 用户接口（作业管理）





3. 操作系统的三种基本类型及特点

(1). 批处理系统

主要特点:

① 脱机操作; ② 成批处理; ③ 多道程序运行; ④ 无交互性。

(2). 分时系统 多用户

主要特点:

① 交互性; ② 同时性; ③ 独立性; ④ 及时性。

(3). 实时系统

主要特点:

① 实时时钟管理; ② 连续人机对话; ③过载防护, 安全可靠; ④ 资源利用率低





4. 多道程序运行及特点

multi-processing

多道程序设计：允许多作业同时进入内存轮流交替占用CPU运行的技术。

特点：（1）多道性

（2）宏观上并行

（3）微观上串行





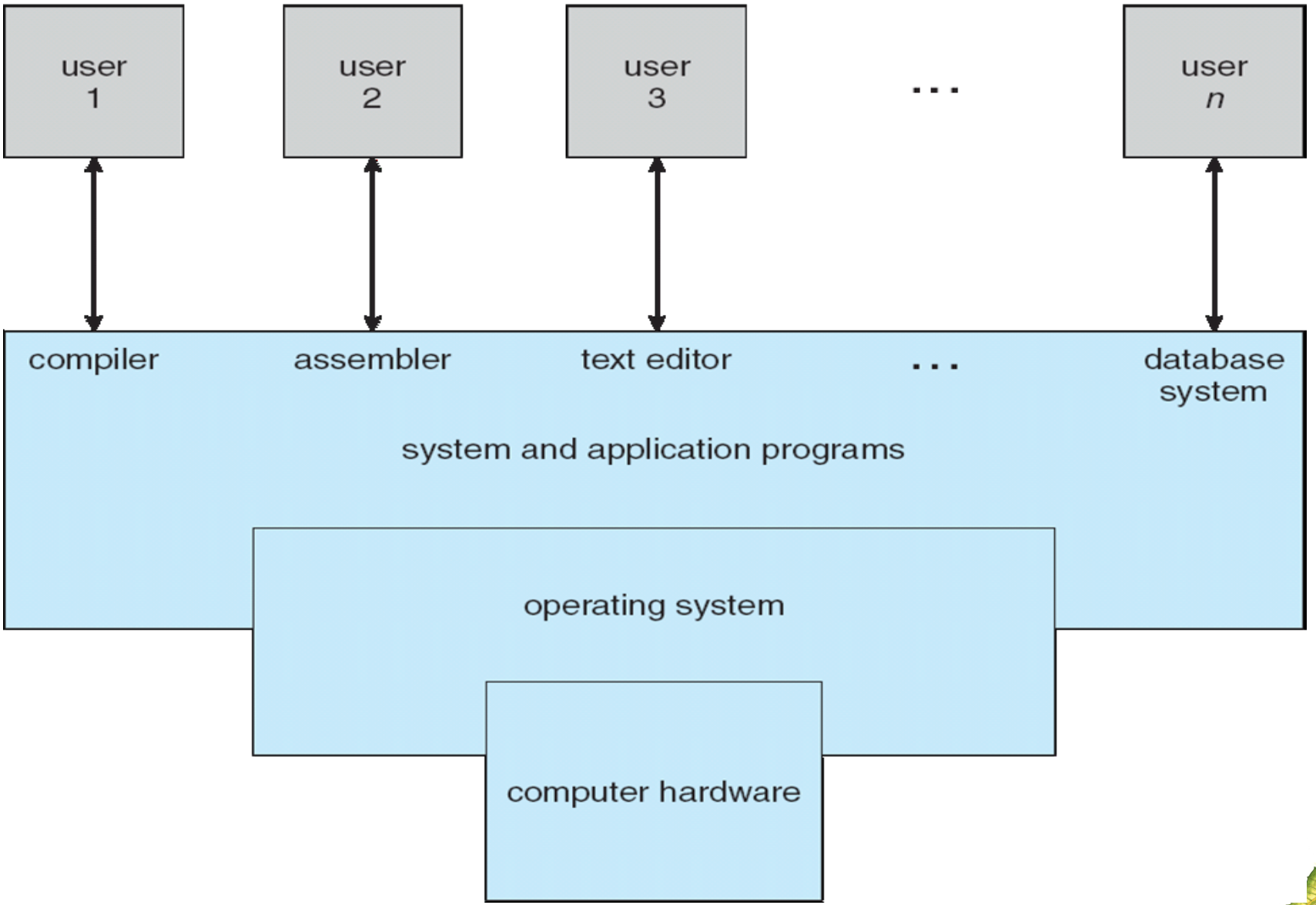
5. Computer System Structure

- Computer system can be divided into four components
  - Hardware – provides basic computing resources
    - CPU, memory, I/O devices
  - Operating system
    - Controls and coordinates use of hardware among various applications and users
  - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
    - Word processors, compilers, web browsers, database systems, video games
  - Users
    - People, machines, other computers





Four Components of a Computer System







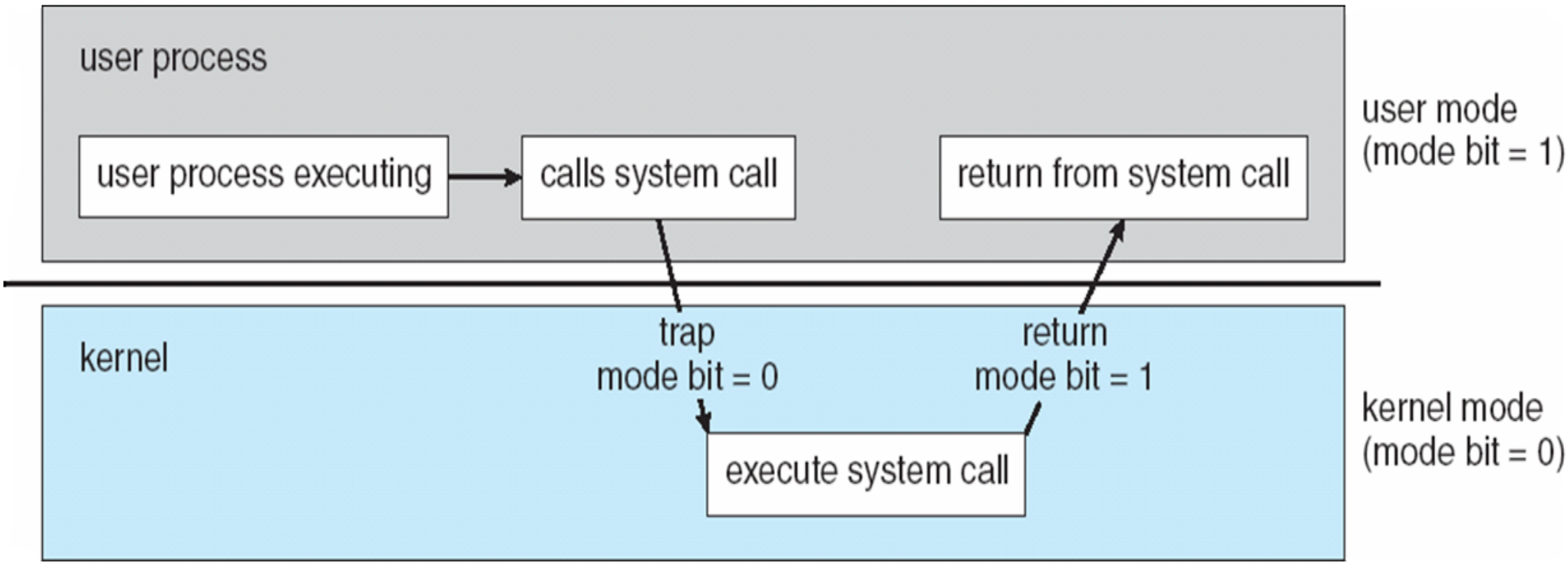
6. User mode and kernel mode

- **Dual-mode operation allows OS to protect itself and other system components**
  - User mode: a user mode process can only execute normal instructions, but not privileged instructions.
  - Kernel mode: a kernel mode process can execute all instructions. Only OS processes should be executing in kernel mode.
    - It is also called system, privileged or supervisor mode.





Transition from User to Kernel Mode







本章疑难点

parallism concurrency

1、并行性与并发性的区别和联系

并行性和并发性是既相似又有区别的两个概念。并行性是指两个或多个事件在同一时刻发生，并发性是指两个或多个事件在同一时间间隔内发生。

在多道程序环境下，并发性是指在一段时间内，宏观上有多个程序同时运行，但在单处理器系统中每个时刻却仅能有一道程序执行，因此微观上这些程序只能分时地交替执行。若在计算机系统中有多个处理器，则这些可以并发执行的程序便被分配到多个处理器上，实现并行执行，即利用每个处理器来处理一个可并发执行的程序。





本章疑难点

kernel instruc

2、特权指令与非特权指令

谓特权指令，是指有特殊权限的指令，由于这类指令的权限最大，使用不当将导致整个系统崩溃，如清内存、置时钟、分配系统资源、修改虚存的段表或页表、修改用户的访问权限等。若所有程序都能使用这些指令，则系统一天死机n次就不足为奇。

为保证系统安全，这类指令只能用于操作系统或其他系统软件，不直接提供给用户使用。因此，特权指令必须在核心态执行。实际上，CPU在核心态下可以执行指令系统的全集。形象地说，特权指令是那些儿童不宜的东西，而非特权指令是老少皆宜的东西。





本章疑难点

system call

3、访管指令与访管中断

访管指令是一条可以在用户态执行的指令。在用户程序中，因要求操作系统提供服务有意识地使用访管指令，从而产生一个中断事件(自愿中断)，将操作系统转换为核心态，称为访管中断。访管中断由访管指令产生，程序员使用访管指令向操作系统请求服务。

为什么要在程序中引入访管指令呢?这是因为用户程序只能在用户态下运行。若用户程序想要完成在用户态下无法完成的工作，该怎么办?解决这个问题要靠访管指令。访管指令本身不是特权指令，其基本功能是让程序拥有“自愿进管”的手段，从而引起访管中断。

处于用户态的用户程序使用访管指令对，系统根据访管指令的操作数执行访管中断处理程序，访管中断处理程序将按系统调用的操作数和参数转到相应的例行子程序。完成服务功能后，退出中断，返回到用户程序断点继续执行。

↓ routine





例题

（ ）操作系统是对（ ）进行管理的软件。

A. 软件 B. 硬件 C. 计算机资源 D. 应用程序

答： C





例题

（ ）操作系统的基本特征不包括（D）。

A. 并发性 B. 共享性 C. 虚拟性 D. 顺序性

答： D





例题

C ( ) 操作系统允许在一台主机上同时连接多台终端，多个用户可以通过各自的终端同时交互地使用计算机

A 网络

B 分布式

C 分时

D 实时

答案： C





例题

单处理机计算机系统中，（ ）是并行操作的。

A.处理机的操作与通道的操作是并行的

B.程序与程序

C.主程序与子程序

D.用户程序与操作系统程序

答案： A





例题

当CPU处于管态时，它可以执行的指令是（ ）。

A.计算机系统中的全部指令

B.仅限于非特权指令

C.仅限于访管指令

D.仅限于特权指令

答案： A





例题

下面6个任务中，必须是实时任务的有（）个

- 航空订票任务
- 过程控制任务
- 机器口语翻译任务
- 计算机辅助任务
- 办公自动化任务
- 计算机激光照排任务

A.1　　B.2　　C.3　　D.4

答案：C





例题

(B) 对多用户分时系统最重要；(D) 对多道批处理系统最重要；

(A)对实时系统最重要。

- A.实时性　　B.交互性
- C.共享性　　D.运行效率

答：B　D　A





例题

在单处理机系统中，相同的硬件条件下，要执行10个程序，每个程序单独执行需要6分钟，现在10个程序同时在多道程序系统执行，一般情况下，每个程序执行完毕需要的时间（B.）），全部执行完毕总共需要的时间（C.）。

A.小于6分钟　　B.大于6分钟

C.小于等于60分钟　D.大于60分钟

答案：B，C





例题

【多选】ABD 不适合批处理。↗需要交互

A.玩游戏　　B.飞行模拟

C.解大型方程组　　D.调试程序

答案： ABD





例题

【多选】下列哪些指令只能在系统态执行？（）A D.

A.关中断　　B.读系统时钟

C.向寄存器存数据　　D.修改系统时钟

答案： AD





例题

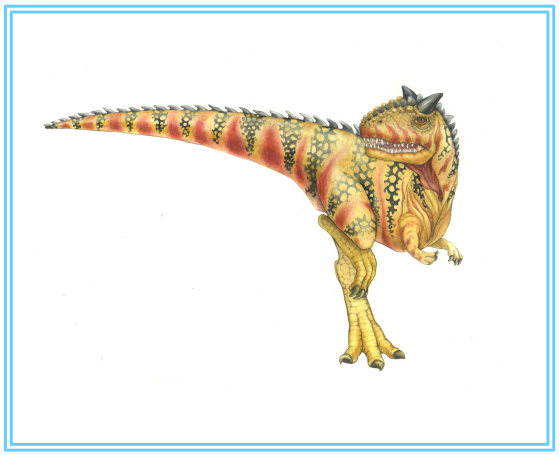
【简答】将手工操作、单道批处理、多道批处理、多用户分时系统按CPU的有效利用率，由小到大进行排列。

答案：手工操作、单道批处理系统、多用户分时系统、多道批处理系统。

(1) 手工操作没有操作系统，属于单道程序系统，大量的处理机时间被人工操作所浪费，因此CPU的利用率最低。
(2) 单道批处理系统在一定程度上克服了手工操作的缺点，但仍属于单道程序系统，大量的CPU时间浪费在等待I/O操作的完成上。因此它的CPU利用率比手工操作的系统要高，但比多道程序系统的要低。
(3)多用户分时系统是多道程序系统，具有交互性，但是程序的分时运行需CPU不断地在多个程序之间进行切换，这种切换需要占用的CPU的时间。
(4)多道批处理系统是多道程序系统，没有交互性。CPU在执行一道程序时一般不转换到其它道程序，只有在需要等待某种事件发生时，才切换到



第2章 操作系统结构



Silberschatz, Galvin and Gagne ©2009



第2章：操作系统结构 概要总结

- 操作系统功能：
  - 服务用户





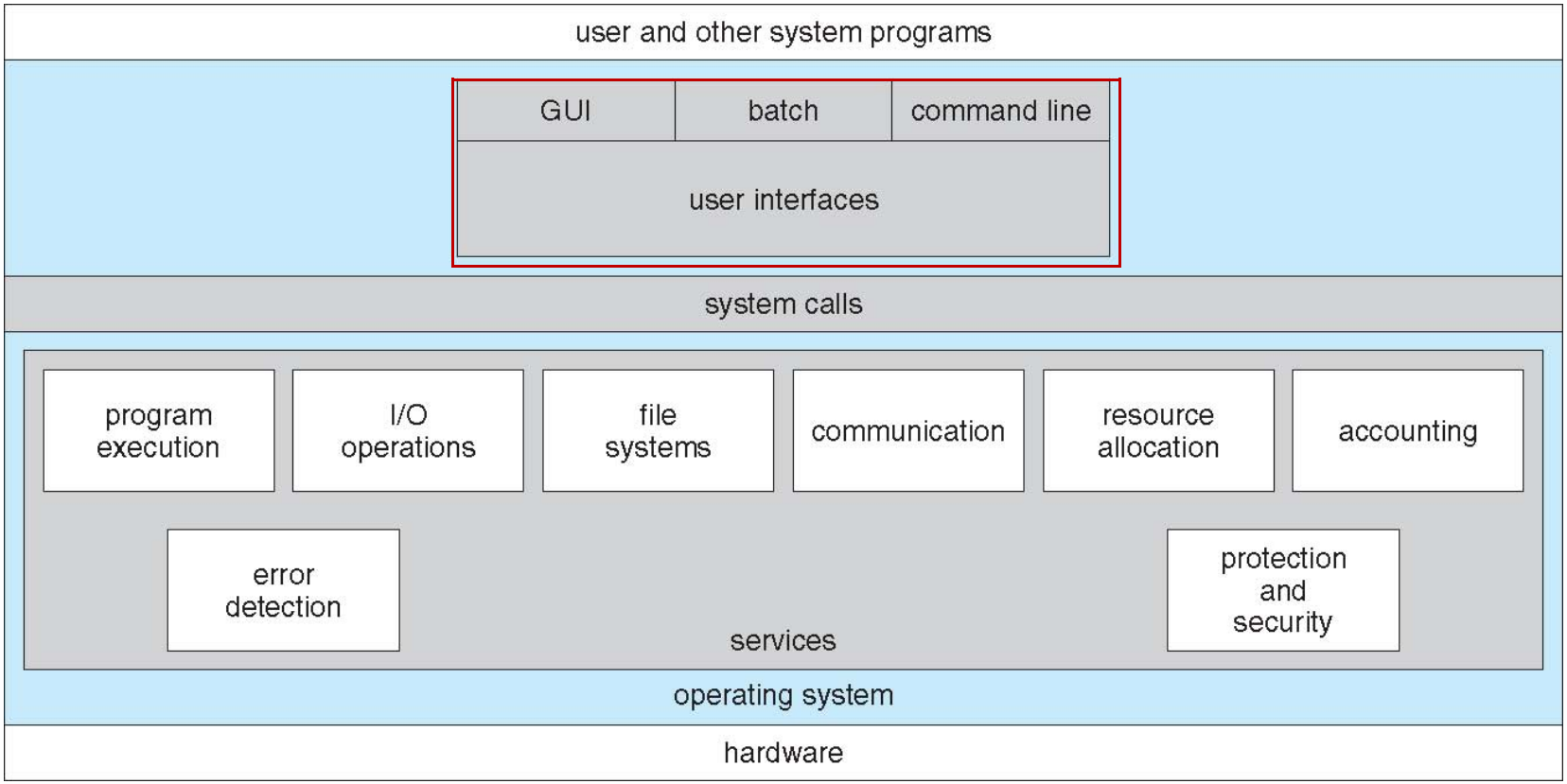
Operating System Services

- We can view OS from several points:
  - The services that the OS provides
  - The interface provided to users and programmers
  - Its components and interconnections





A View of Operating System Services







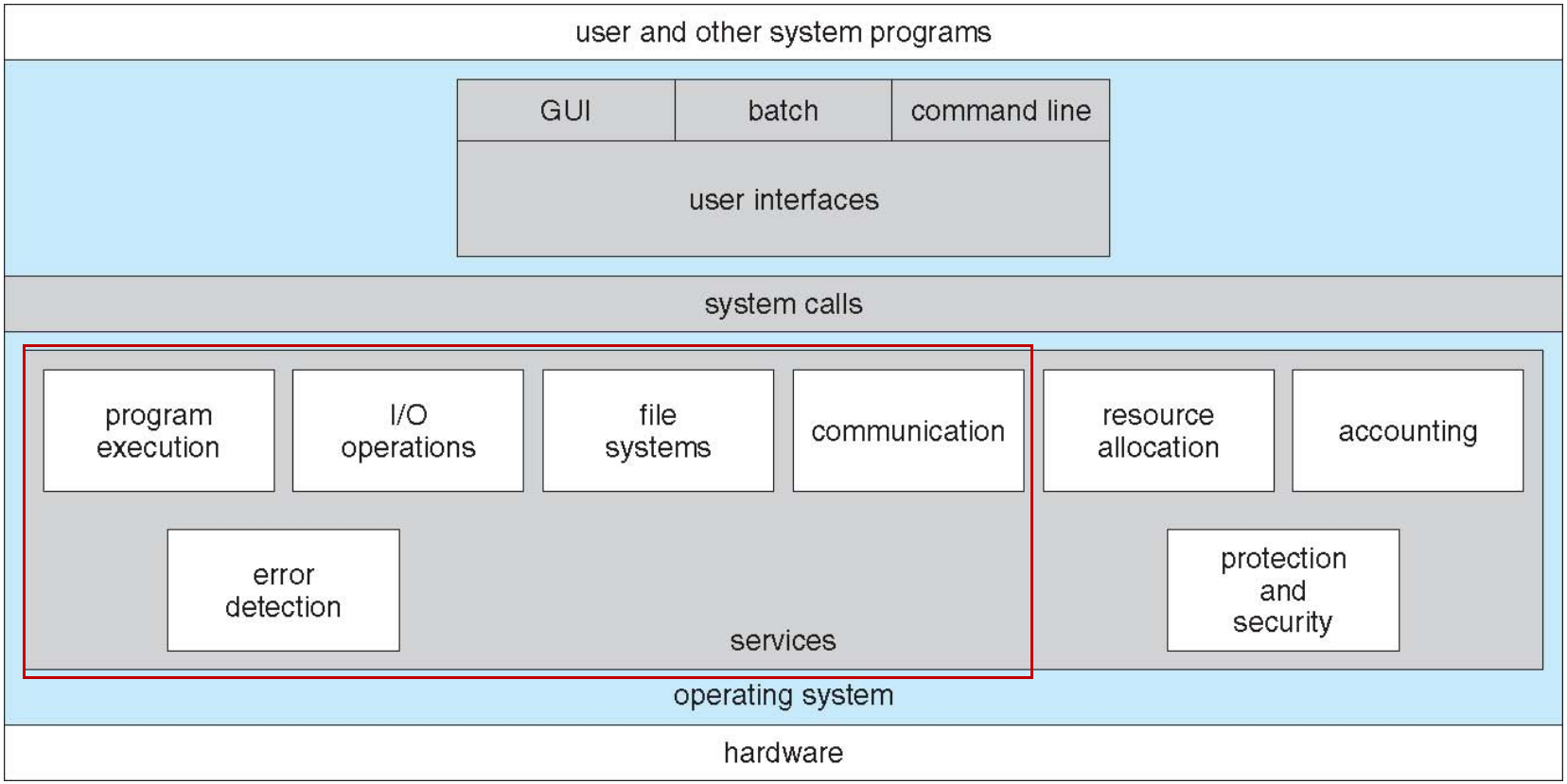
User Operating System Interface

- Command interface
  - Command Line Interface (CLI) or command interpreter (命令行)
  - GUI interface（用户界面）
  - Batch（批处理）
- Program interface
  - System call





A View of Operating System Services







Services to the user (Cont)

- Program execution（程序执行）
  - The system must be able to load a program into memory and to run that program, end execution, either normally or abnormally (indicating error)
- I/O operations（I/O操作）
  - A running program may require I/O, which may involve a file or an I/O device
- File-system manipulation （文件系统操作）
  - Obviously, programs need to read and write files and directories, create and delete them, search them, list file Information, permission management.





Services to the user (Cont)

- Communications （进程通信）
  - Processes may exchange information, on the same computer or between computers over a network
    - Communications may be via shared memory or through message passing (packets moved by the OS)





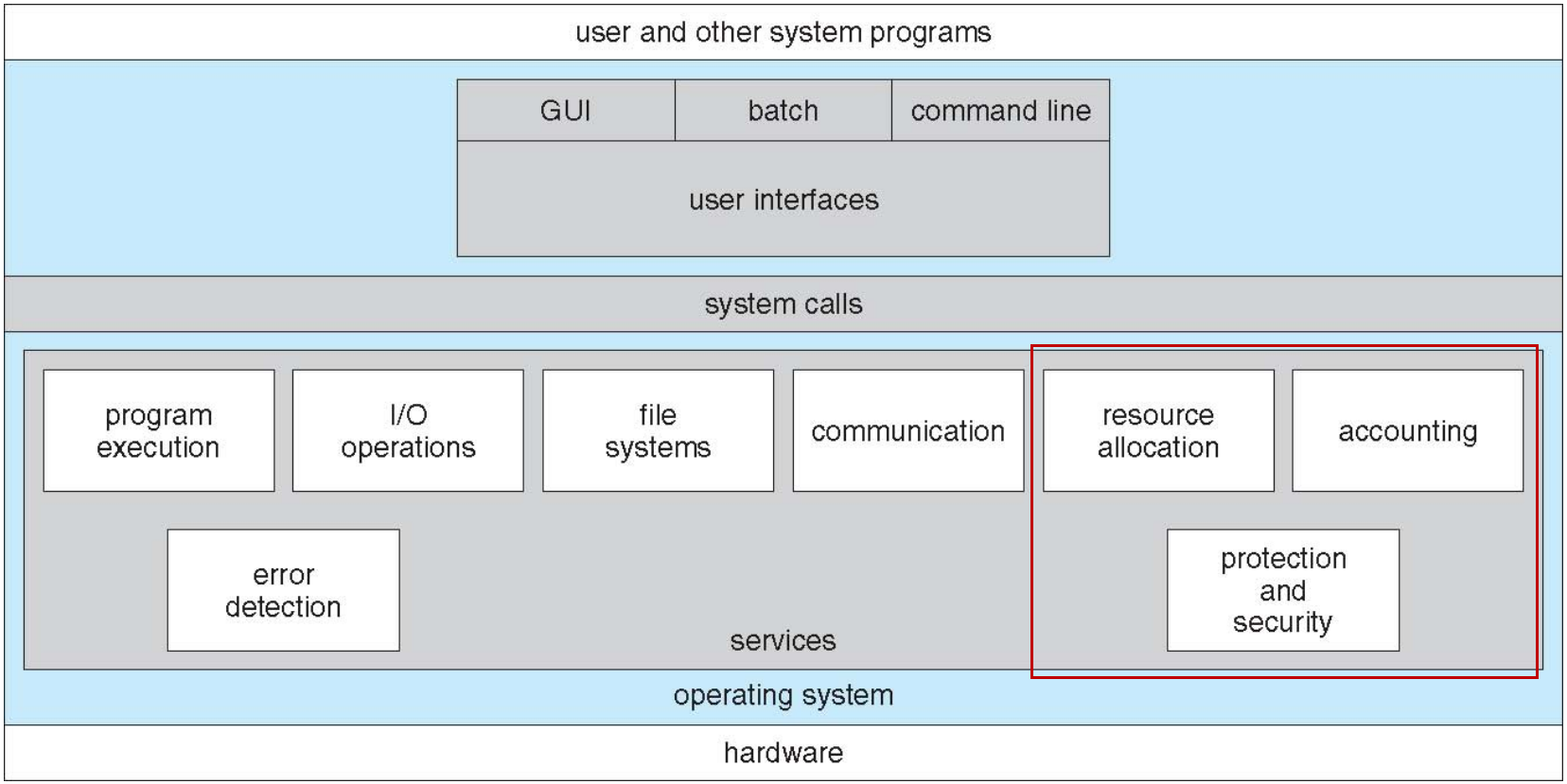
Services to the user (Cont)

- Error detection – OS needs to be constantly aware of possible errors
  - May occur in the CPU and memory hardware, in I/O devices, in user program
  - For each type of error, OS should take the appropriate action to ensure correct （正确） and consistent （一致） computing
  - Debugging facilities can greatly enhance the user's and programmer's abilities to efficiently use the system





A View of Operating System Services







Services for the System Itself

- **Resource allocation（资源分配） -** When multiple users or multiple jobs running concurrently, resources must be allocated to each of them
  - Many types of resources - Some (such as CPU cycles, main memory, and file storage) may have special allocation code, others (such as I/O devices) may have general request and release code
- **Accounting （统计） -** To keep track of which users use how much and what kinds of computer resources



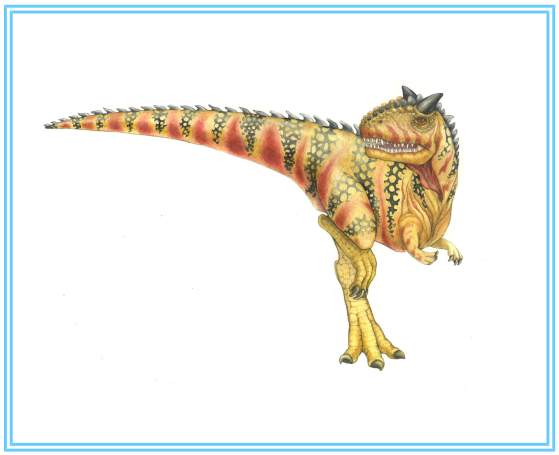


Services for the System Itself (Cont)

- **Protection and security（保护和安全） -** The owners of information stored in a multiuser or networked computer system may want to control use of that information, concurrent processes should not interfere with each other
  - **Protection** involves ensuring that all access to system resources is controlled
  - **Security** of the system from outsiders requires user authentication, extends to defending external I/O devices from invalid access attempts



第3章 进程



Silberschatz, Galvin and Gagne ©2009



第3章：进程 概要总结

- 进程概念：
  - 进程与程序的区别与联系
  - 进程与线程的区别与联系
  - 子进程与父进程的关系
- 进程控制块（PCB）
- 进程状态转换（五种状态在何种情况下发生转换）
- 进程通信
  - 共享内存
  - 消息传递

Ready

Running

Block





Process Concept

进程是动态的，程序是静态的：程序是有序代码的集合；进程是程序的执行。

进程是暂时的，程序是永久的：进程是一个状态变化的过程，程序可长久保存。

进程与程序的组成不同：进程的组成包括程序、数据和进程控制块（即进程状态信息）。

进程与程序的对应关系：通过多次执行，一个程序可对应多个进程；通过调用关系，一个进程可对应多个程序。





Process Concept

结构特征：进程实体=程序段+相关的数据段+PCB。 code, data

动态性：进程的实质是进程实体的一次执行过程，因此动态性是进程的最基本的特征。

并发性：多个进程实体同存在于内存中，且能在一段时间内同时运行。是最重要的特征。

独立性：进程实体是一个能独立运行、独立分配资源和独立接受调度的基本单位。

异步性：进程按各自独立的、不可预知的速度向前推进。





1. 程序顺序执行及特点

程序在处理机上的执行是严格按序的。

特点：

① 顺序性

② 封闭性

③ 可再现性





2. 进程并发执行及特点

进程在处理机上的执行时间是交叉重叠的，是提高CPU利用率而采取的一种同步操作技术。

特点：

① 独立性

② 随机性

③ 资源共享性





3. 进程的定义及引入目的

定义 一个具有独立的功能的程序关于某个数据集在处理机上的一次执行过程及分配资源的基本单位。

引入目的

① 为了控制和协调并发程序对软硬件资源的共享和竞争。

② 为了描述程序动态执行的过程和有个分配资源的基本单位。





5. 进程的描述

进程的描述包括三部分：

① 程序

② 数据结构集

③ 进程控制块（PCB）





6. 进程的状态及转换

① 运行状态

一个进程正占用CPU执行。

② 等待状态

进程因等待某事件不能享用CPU.

③ 就绪状态

进程已具备运行条件尚未占用CPU。

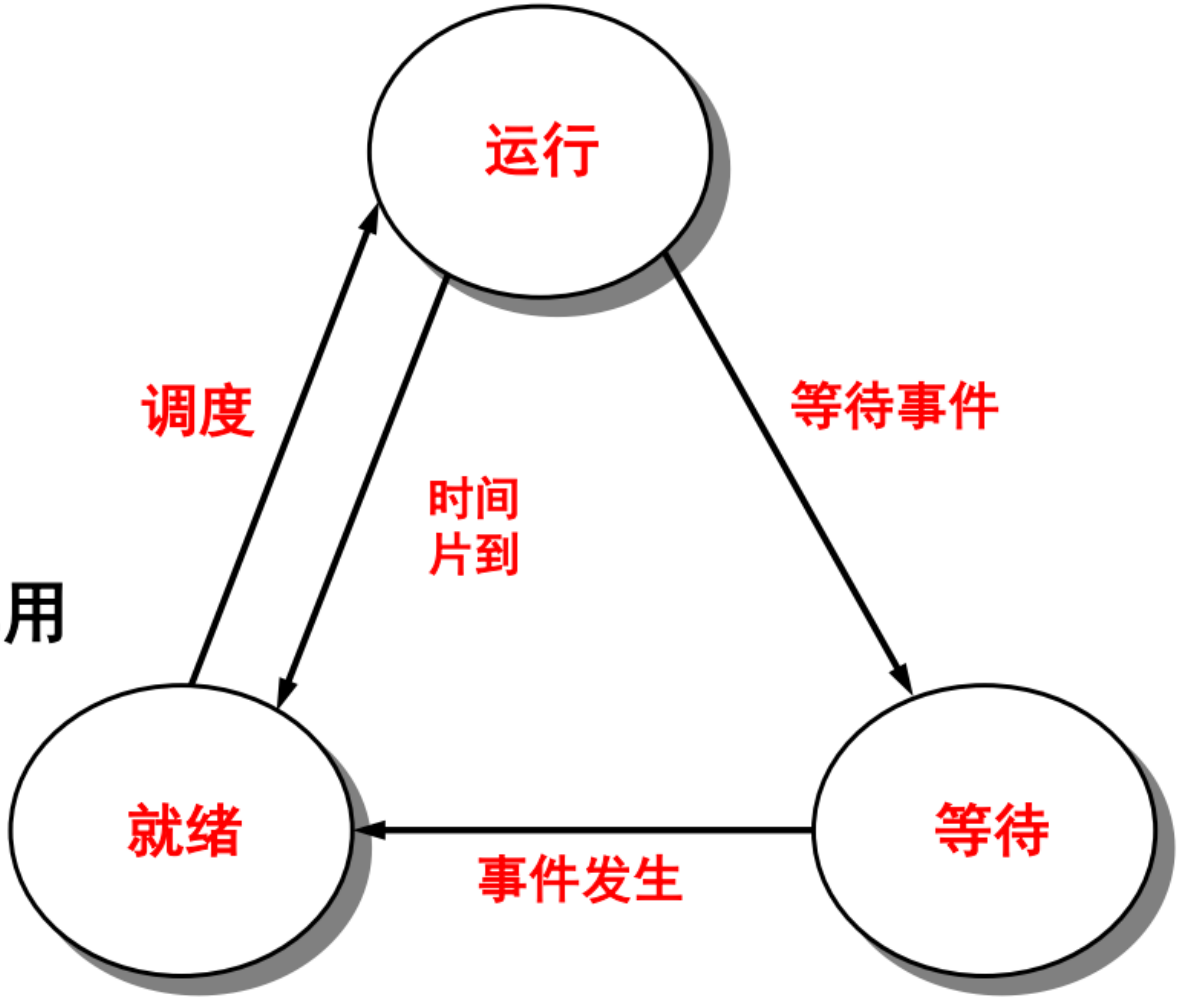
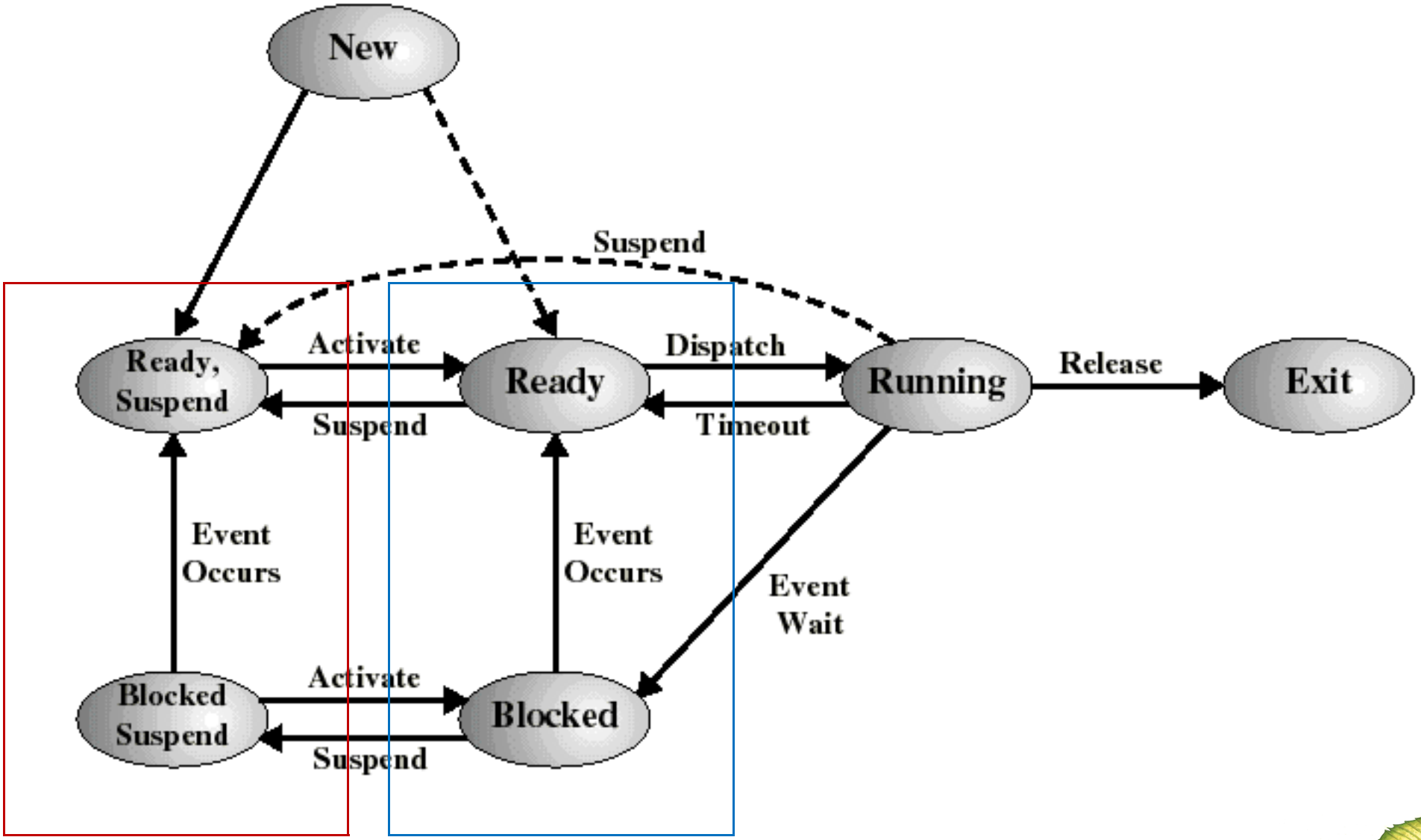






Diagram of Process State







Process Control Block (PCB)

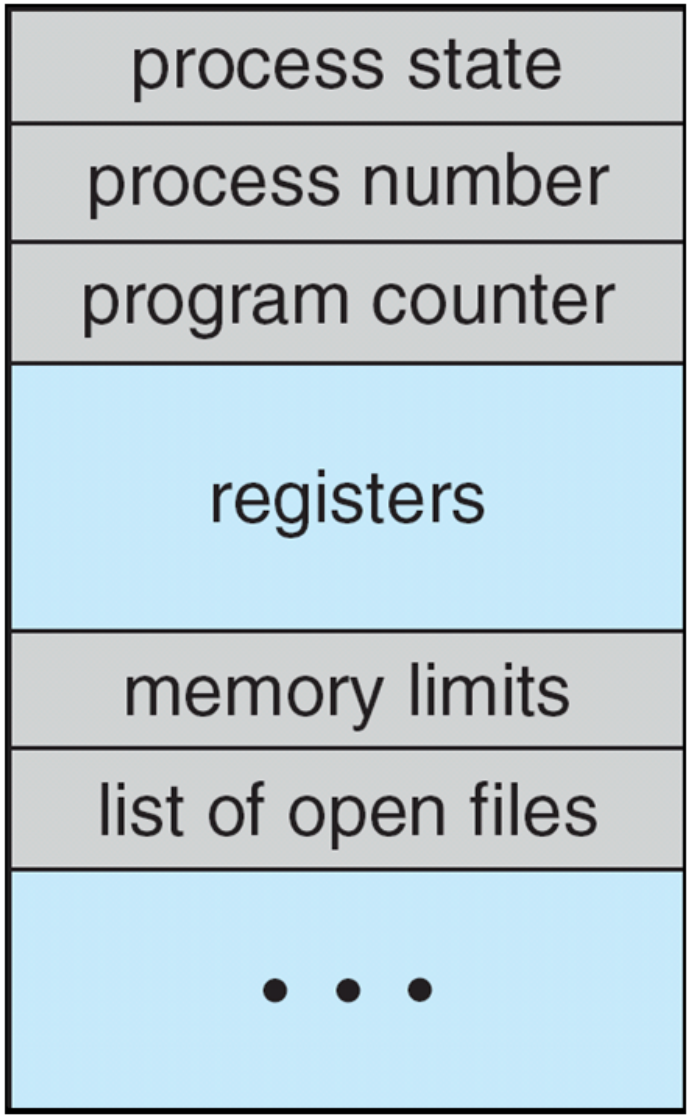
Information associated with each process

- Process state
- Program counter
- CPU registers
- CPU scheduling information
- Memory-management information
- Accounting information
- I/O status information





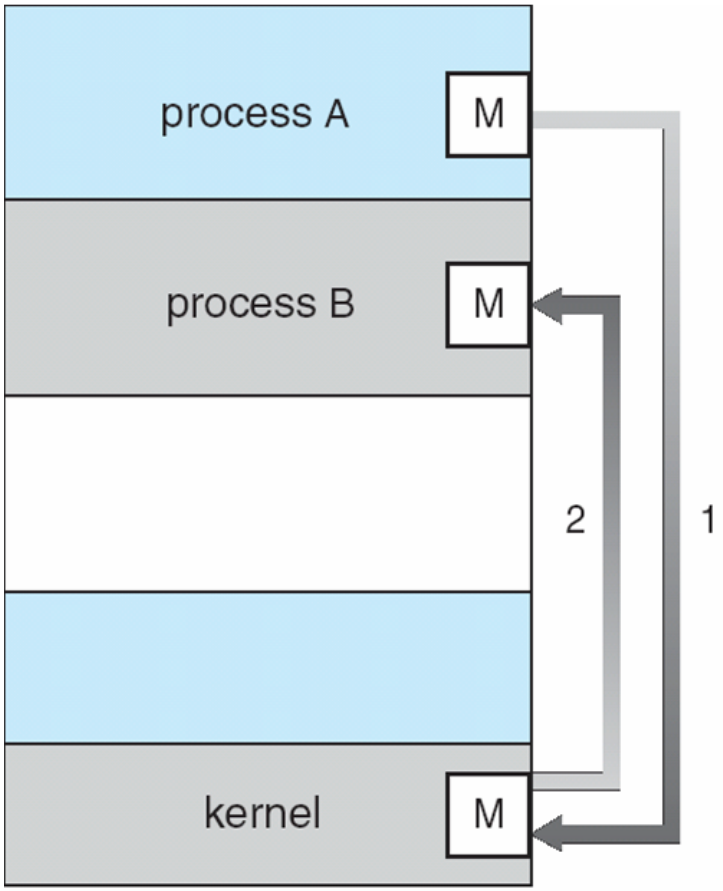
Process Control Block (PCB)





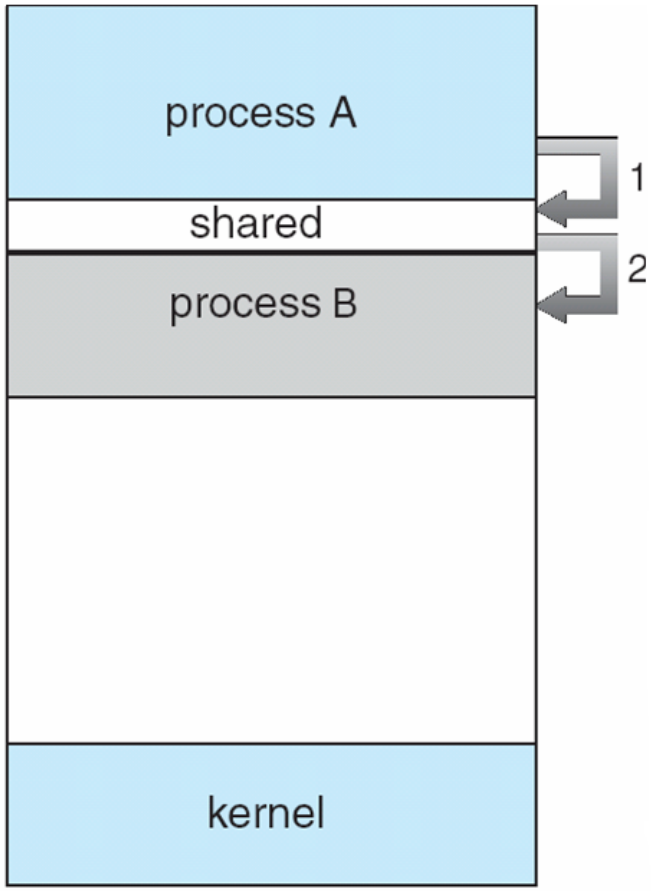


Communications Models



(a)

Communicating



(b)

Sharing





例题

在操作系统中，进程的最基本特征是（）

A 动态性和并发性

B 顺序性和可再现性

C 与程序的对应性

D 执行过程的封闭性

答案： A





例题

进程的共享性主要体现在（）

a.处理器； b.主存； c. 外存； d.数据

A (a, b)

B (a,b,c)

C (a,b,c,d)

D (a,b,d)

答案： C





例题

下列有可能导致一进程从运行变为就绪的事件是（D）。

A.一次I/O操作结束

B.运行进程需作I/O操作

C.运行进程结束

D.出现了比现运行进程优先权更高的进程

答案： D





例题

suspend → disk

【多选】在（）时，可能挂起某进程。

A.进程出现死锁 ✓　B.进程的数目太少

C.进程数目太多 ✓　D.进程出现故障 ✓

答案： ACD

resources in memory run out.





例题

【多选】下面会引起进程创建的事件是（）。

A.用户登录 √ B. 设备中断

C.作业调度 √ D. 执行系统调用 √

↳ disk to memory

答案： ACD





例题

【简答】PCB有什么作用？为什么说PCB是进程存在的唯一标识？

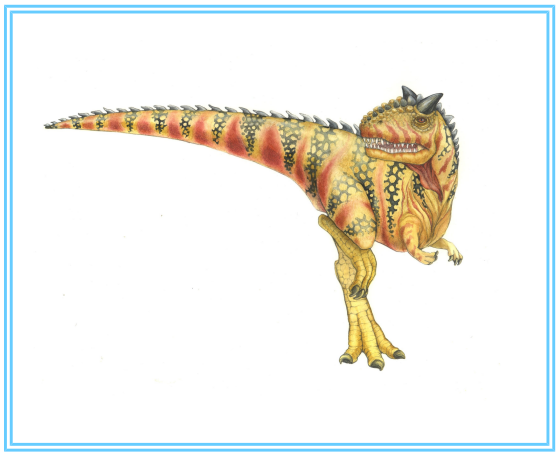
答案：

PCB的作用：操作系统调度进程的主要数据依据，记录进程的有关信息，供系统对进程进行控制；标识进程的存在

唯一标识：在进程的整个生命周期中，系统总是通过PCB对进程进行控制，系统是根据进程的PCB感知到该进程的存在的



第4章 多线程编程



Silberschatz, Galvin and Gagne ©2009



第4章 多线程 概要总结

- 线程概念：“轻量级”进程
- 内核级线程 vs. 用户级线程
  - 对应关系





Threads and Processes

- Most operating systems therefore support two entities:
  - the process,
    - which defines the address space and general process attributes
  - the thread,
    - which defines a sequential execution stream within a process
- Threads are the unit of scheduling
- Processes are *containers* in which threads execute





Implementation of thread

Kernel-supported threads

Supported directly by the operating system

User-level threads

supported above the kernel, via a set of library calls at the user level

pthread





User-Level vs. Kernel Threads

User-Level

- Managed by application
- Kernel not aware of thread
- Context switching cheap
- Create as many as needed
- Must be used with care

Kernel-Level

- Managed by kernel
- Consumes kernel resources
- Context switching expensive
- Number limited by kernel resources
- Simpler to use

Key issue: kernel threads provide virtual processors to user-level threads, but if all of kthreads block, then all user-level threads will block *even* if the program logic allows them to proceed





Multithreading Models

- Many-to-One
- One-to-One
- Many-to-Many

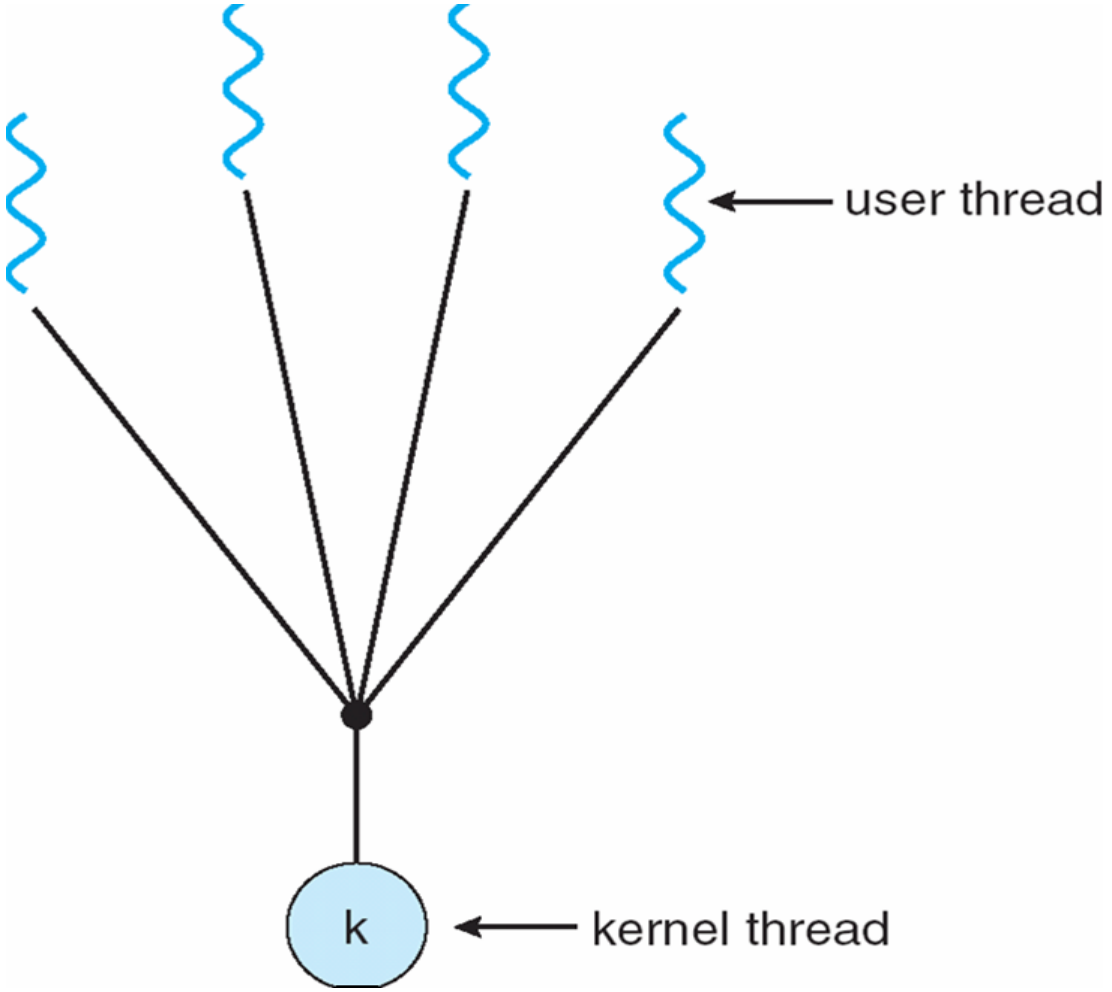
- Two level model





Many-to-One Model

Many user-level threads mapped to single kernel thread

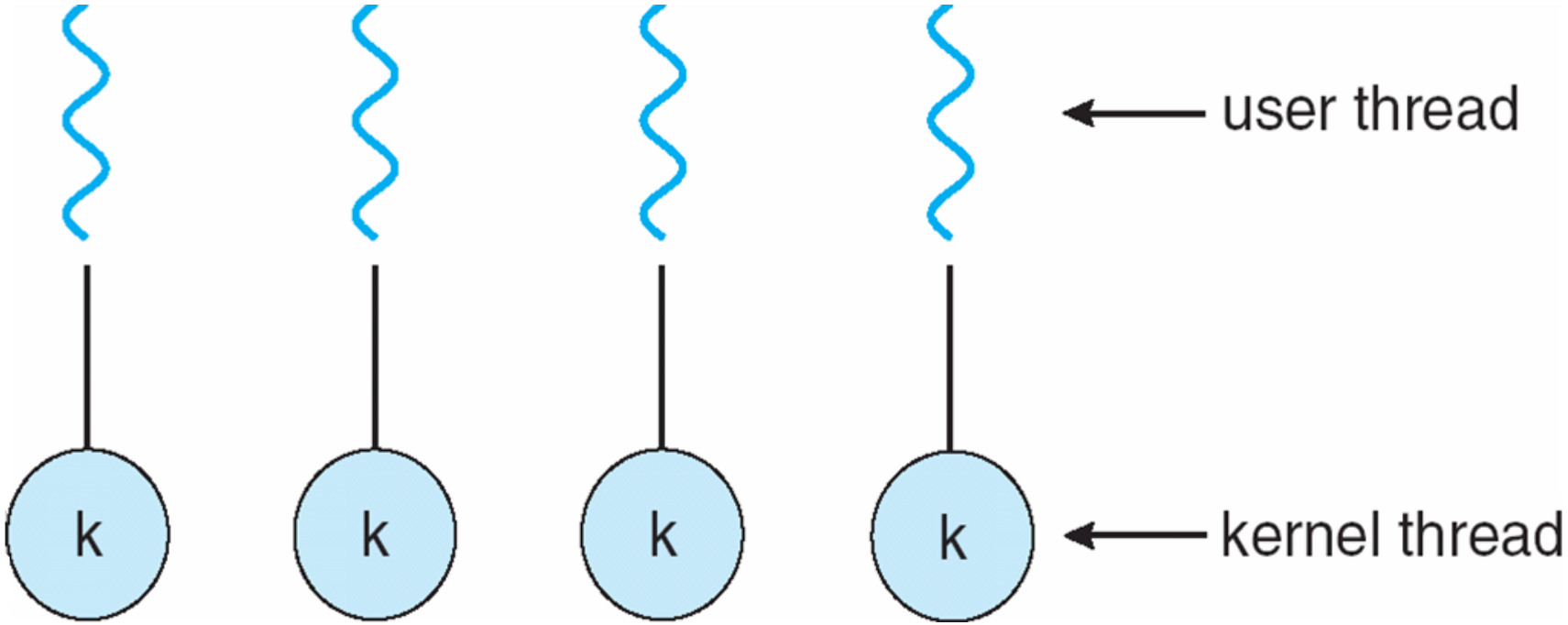


- No parallel execution of threads - can't exploit multiple CPUs
- All threads block when one uses synchronous I/O





One-to-one Model

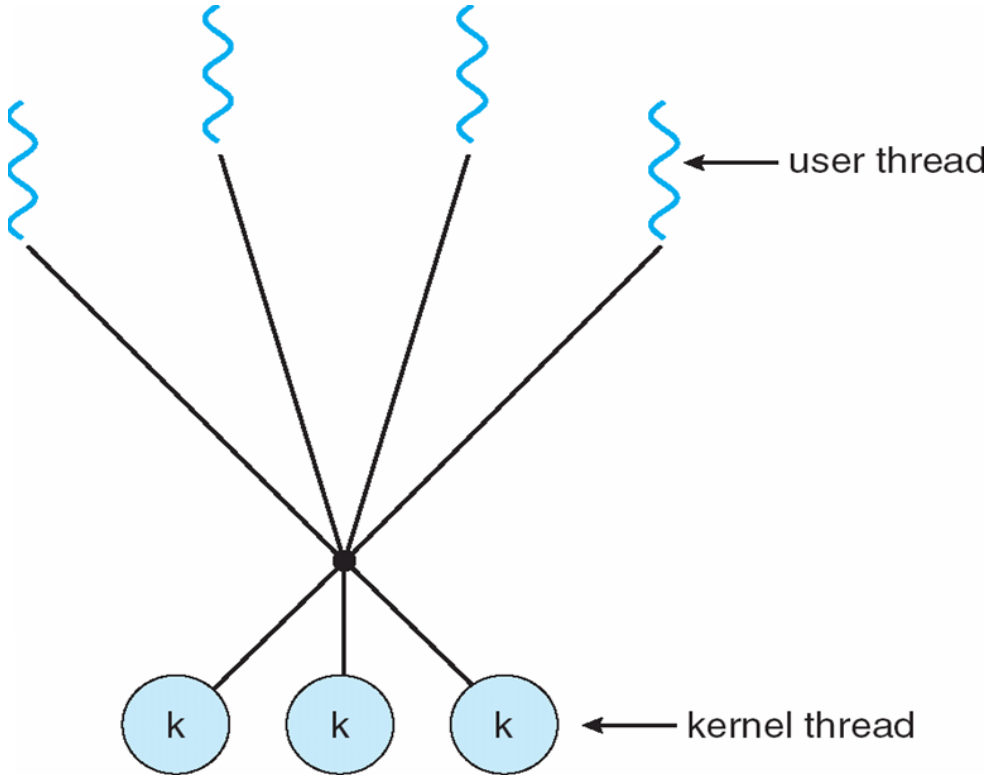


- More concurrency
- Better multiprocessor performance
- Each user thread requires creation of kernel thread
- Each thread requires kernel resources; limits number of total threads





Many-to-Many Model



If U<k? No benefits of multithreading

If U>k, some threads may have to wait for an Kthread to run

- Active thread - executing on an Kthread
- Runnable thread - waiting for an Kthread

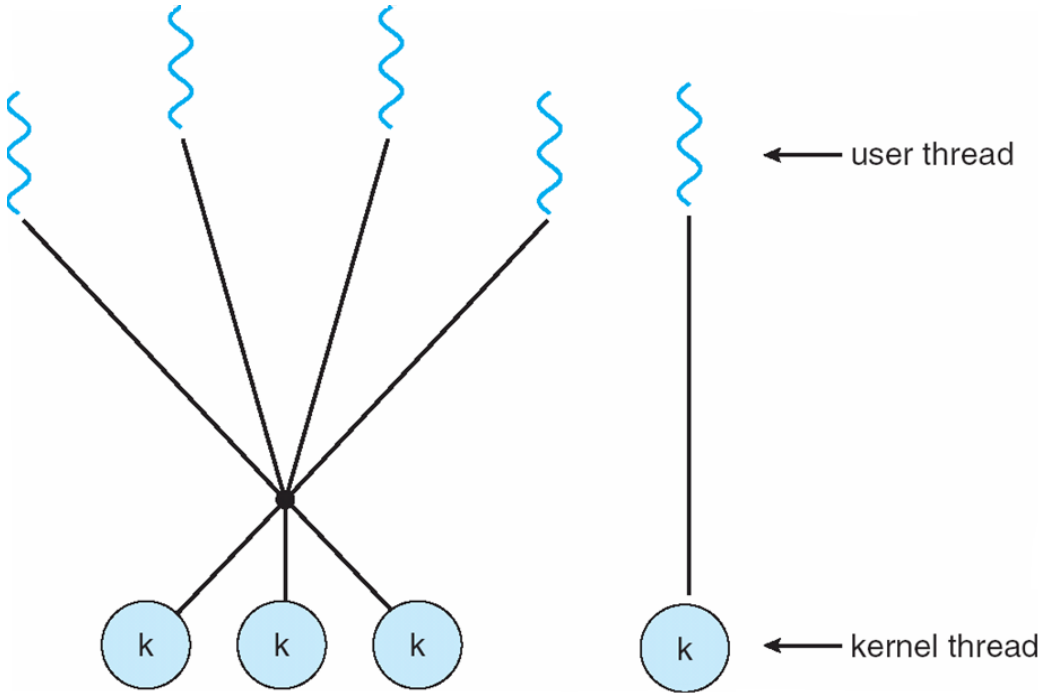
A thread gives up control of Kthread under the following:

– synchronization, lower priority, yielding, time slicing





Two-level Model



- Supports both bound and unbound threads
- Bound threads - permanently mapped to a single, dedicated kthread
- Unbound threads - may move among kthreads in set
- Thread creation, scheduling, synchronization done in user space





例题

下面关于线程的叙述中，正确的是（ ）。

A.不论是系统支持线程还是用户级线程，其切换都需要内核的支持。

B.线程是资源的分配单位，进程是调度和分配的单位。

C.不管系统中是否有线程，进程都是拥有资源的独立单位。

D.在引入线程的系统中，进程仍是资源分配和调度分派的基本单位。





例题

下列几种关于进程的叙述，（A）最不符合操作系统对进程的理解？

A.进程是在多程序并行环境中的完整的程序。

B.进程可以由程序、数据和进程控制块描述。

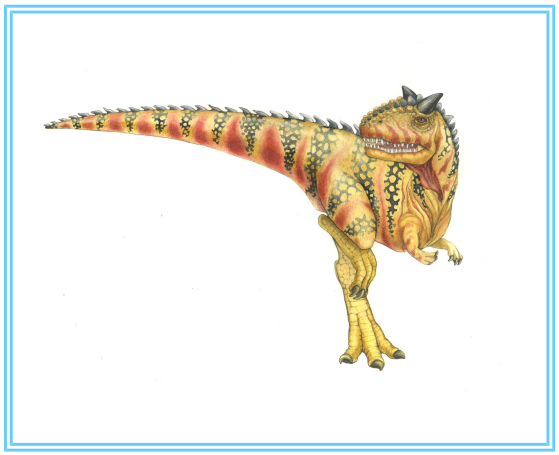
C.线程是一种特殊的进程。

D.进程是程序在一个数据集合上运行的过程，它是系统进行资源分配和调度的一个独立单位。

答案： A



第5章 进程调度



Silberschatz, Galvin and Gagne ©2009



第5章 进程调度 概要总结

- 进程调度的时机
- 进程调度时的状态转变
- 进程调度算法
  - 算法评估指标：吞吐率、响应时间、周转时间、等待时间
  - 进程调度算法
  - 作业调度（调入内存）+进程调度（调度CPU）





3. 引起进程调度的时机

（1）正在执行的进程执行完毕。这时，如果不选择新的就绪进程执行，将浪费处理机资源。

（2）执行中进程自己调用阻塞原语将自己阻塞起来进入睡眠等待状态。

（3）执行中进程调用了P原语操作，从而因资源不足而被阻塞；或调用了V原语操作激活了等待资源的进程队列。

（4）执行中的进程提出I/O请求后被阻塞。

（5）在分时系统中时间片已经用完。

（6）在执行完系统调用，在系统程序返回用户进程时，可认为系统进程执行完毕，从而可调度选择一新的用户进程执行。

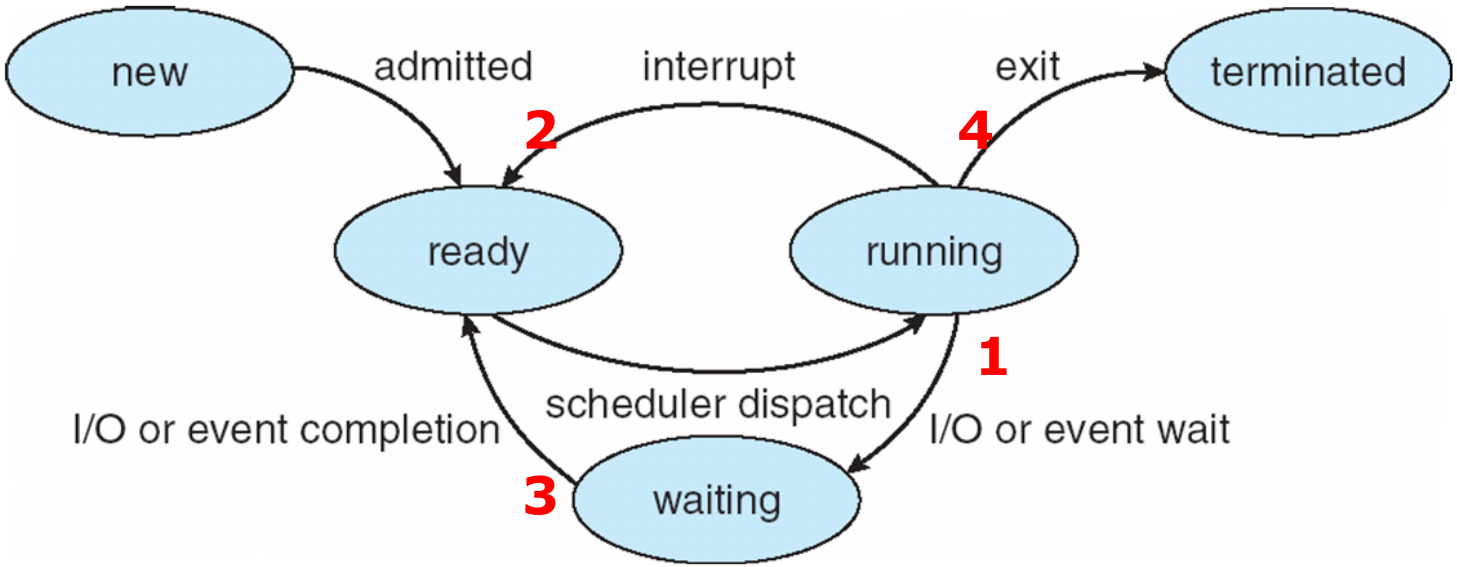
（7）就绪队列中的某进程的优先级变得高于当前执行进程的优先级，从而也将引发进程调度。





CPU Scheduler

- CPU scheduling decisions may take place when a process:
  1. Switches from running to waiting state
  2. Switches from running to ready state
  3. Switches from waiting to ready
  4. Terminates



- Scheduling
- All other scheduling is **preemptive**





CPU Scheduling scheme

- Nonpreemptive Scheduling（非抢占）
  - Once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state.
- Preemptive scheduling（抢占）
  - According to some rule, the CPU utilization of a process can be preempted .
    - Timer
    - Priority
    - SJF

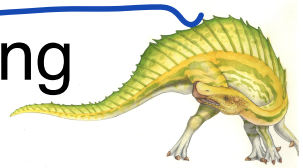
RR

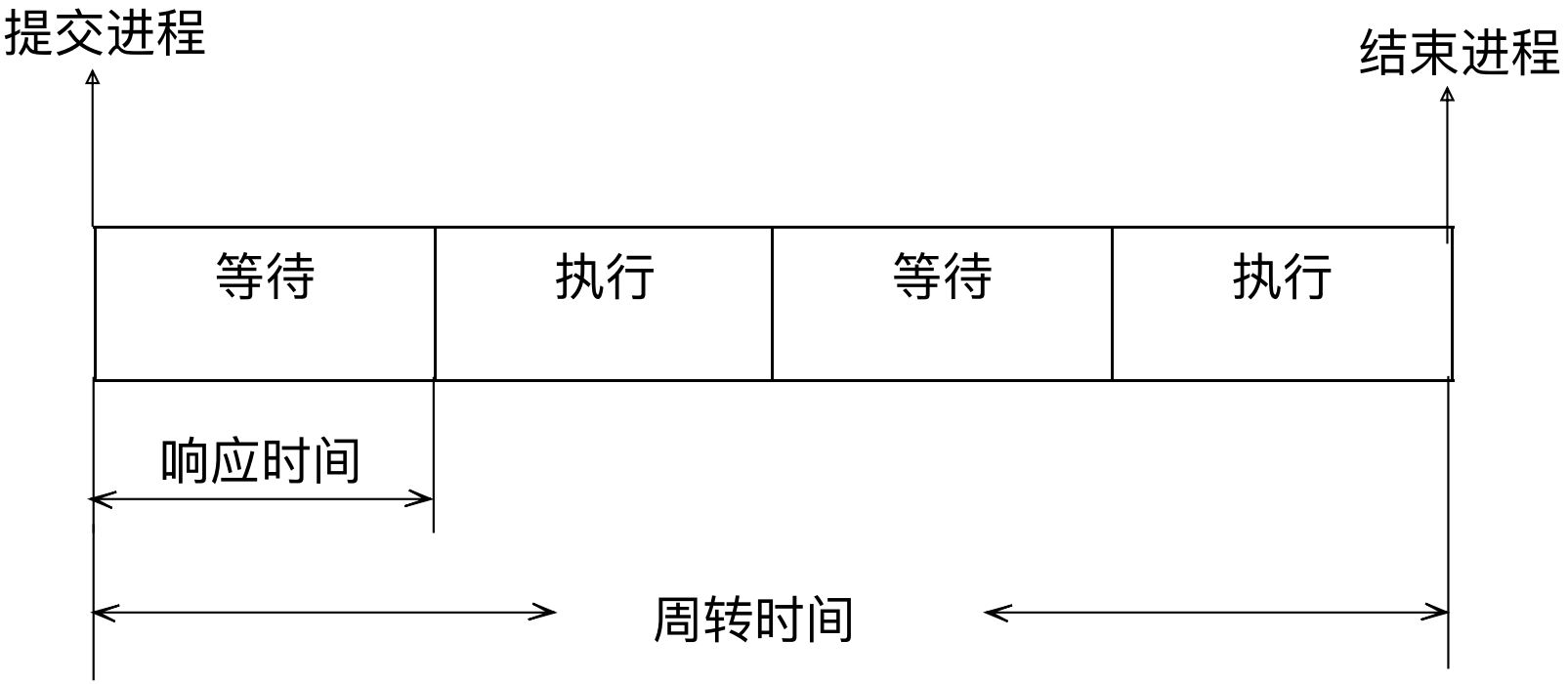




Scheduling Criteria

- **CPU utilization（CPU利用率）** – keep the CPU as busy as possible
- **Throughput （吞吐量）** – # of processes that complete their execution per time unit
- **Turnaround time（周转时间）** – amount of time to execute a particular process
- **Waiting time（等待时间）** – amount of time a process has been waiting in the ready queue
- **Response time（响应时间）** – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)




提交进程
结束进程
等待
执行
等待
执行
响应时间
周转时间



Scheduling Algorithm

- FCFS（先来先服务）
- SJF（最短作业优先）
- Priority scheduling（优先级调度）
- Round Robin (RR，轮转角度)
- Multilevel Queue（多级队列）
- Multilevel Feedback Queue（多级反馈队列）



调度算法	是否抢占	方法	特性
FCFS（先来先服务）	非抢占	先请求CPU的进程首先分配到CPU	平均等待时间往往很长
SJF（最短作业优先）	抢占/ 非抢占	具有更短CPU执行的进程先执行	1. SJF是最优的 2. 困难的是如何知道下次CPU执行的程度
Priority scheduling（优先级调度）	抢占/ 非抢占	具有最高优先级的进程会分配到CPU	1. 问题是无穷阻塞或饥饿 2. 解决方案是老化
Round Robin（轮转调度）	抢占	将一个较小时间单元定义为时间量/时间片，将定时器设置在一个时间片后中断	1.专门为分时系统设计 2.时间片应远大于上下文切换时间
Multilevel Queue（多级队列）	看队列调度算法	将就绪队列分为多个单独队列，每个队列有自己的调度算法	1.前台进程可能要有更高的优先级 2. 进程永久分到一个队列
Multilevel Feedback Queue（多级反馈队列）	抢占	允许进程在队列之间迁移	较低优先级队列中等待过程的进程会被移



例题

【2012】若某单处理器多进程系统中有多个就绪态进程,则下列关于处理机调度的叙述中,错误的是()。

A.在进程结束时能进行处理机调度

B.创建新进程后能进行处理机调度

C.在进程处于临界区时不能进行处理机调度

D.在系统调用完成并返回用户态时能进行处理机调度

答案： C





例题

【2021】下列事件中,可引起进程调度程序执行的是()

I.中断处理结束

II.进程阻塞

III.进程执行结束

IV.进程的时间片用完

A.仅 I、III

B.仅 II、IV

C.仅 III、IV

D.I、II、III、IV





例题

【2009】下列进程调度算法中,综合考虑进程等待时间和执行时间的是(D)。

A.时间片轮转调度算法 ×

B.短进程优先调度算法 ×

C.先来先服务调度算法 ×

D.高响应比优先调度算法

答案：D

响应比：（1+等待时间/执行时间）





例题

【2010】下列选项中,降低进程优先级的合理时机是(A)。

A.进程的时间片用完

B.进程刚完成 I/O,进入就绪队列

C.进程长期处于就绪队列中

D.进程从就绪态转为运行态

答案： A





例题

【2011】下列选项中,满足短任务优先且不会发生饥饿现象的调度算法是(　)。

A.先来先服务

B.高响应比优先

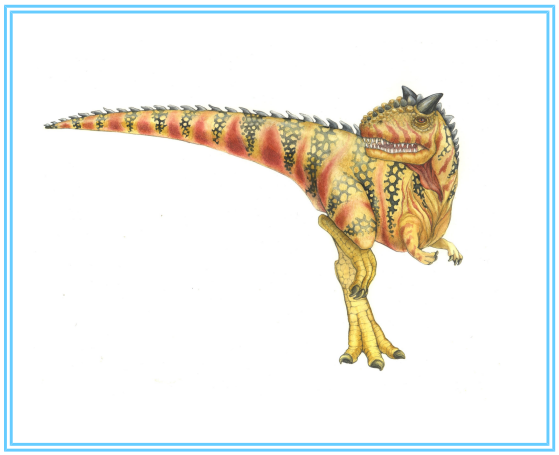
C.时间片轮转

D.非抢占式短任务优先

答案：B



第6章 同步



Silberschatz, Galvin and Gagne ©2009



第6章 同步与互斥 概要总结

- **临界区问题：** critical section.
  - 什么是临界区？（一段操作临界资源的代码）
  - 为什么需要临界区？（互斥访问）
- **各类解决方案：**
  - Peterson （软件）
  - 互斥锁（硬件） mutex
  - 信号量（PV操作）：可以计数 semaphore
  - 管程：高级语言工具，可以定义变量和函数 monitor
- **经典同步问题**
  - 有界缓冲问题（典型案例：生产者-消费者）
  - 读者-作者问题：对临界资源的互斥要求不一样
  - 哲学家就餐问题：为多个进程分配多个资源





Critical-Section

- To ensure the shared data to be modified mutual excluded, no more than one process can execute in its critical section at the same time.
- Each process must request permission to enter its critical section----entry section（进入区）、exit section（退出区）
- The remaining code is the remainder section（剩余区）.

```
entry section
    critical section
exit section
    remainder section
```





7. 临界区与管理原则

临界区：不允许多个并发进程交叉执行的程序段。

管理原则

① 每次至多一个进程进入临界区；

② 进程不能无限期留在临界区；

③ 进程不能相互阻塞。





Solution to Critical-Section Problem

1. Mutual Exclusion 互斥(safety) - No more than one process can be in a critical section at any time.
2. Progress 有空让进(liveness) - If no process is executing in its critical section and there exist some processes wishing to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. Bounded Waiting 有限等待- A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted

- Assume that each process executes at a nonzero speed
- No assumption concerning relative speed of the N processes

n Ideally we would like fairness（公平性） as well

l If two threads are both trying to enter a critical section, they have equal chances of success

l … in practice, fairness is rarely guaranteed





临界区小结

锁是解决临界区问题的常用方法：

互斥可以使用锁来实现

通常需要一定等级的硬件支持

常用的三种实现方法：

禁用中断（仅限于单处理器）

软件方法（复杂）

硬件方法（原子操作指令，单处理器或多处理器均可）

可选的实现内容：

有忙等待

无忙等待





忙等待和无忙等待

- 忙等待

```
AcquireLoc(){

 while(TestandSet(&lock))
}
临界区
Release(){
     *lock=false;
}
```





忙等待和无忙等待

- 无忙等待

```
AcquireLoc(){
        while(TestandSet(&lock))
        add this TCB to wait queue q;
}
临界区
Release() (){
      *lock=false;
      remove one thread t from q;
 wakeup(t)
}
```





9. 进程的同步与互斥

进程同步 相互合作的并发进程之间在某些点要相互通信，互相协调，共同完成任务的过程。

进程互斥 不允许两个或两个以上的并发进程同时进入临界区。





(互斥) Mutual exclusion using semaphore

Semaphore mutex ; // initialized to 1

```
P(mutex);
    critical section
V(mutex);
    remainder section
```

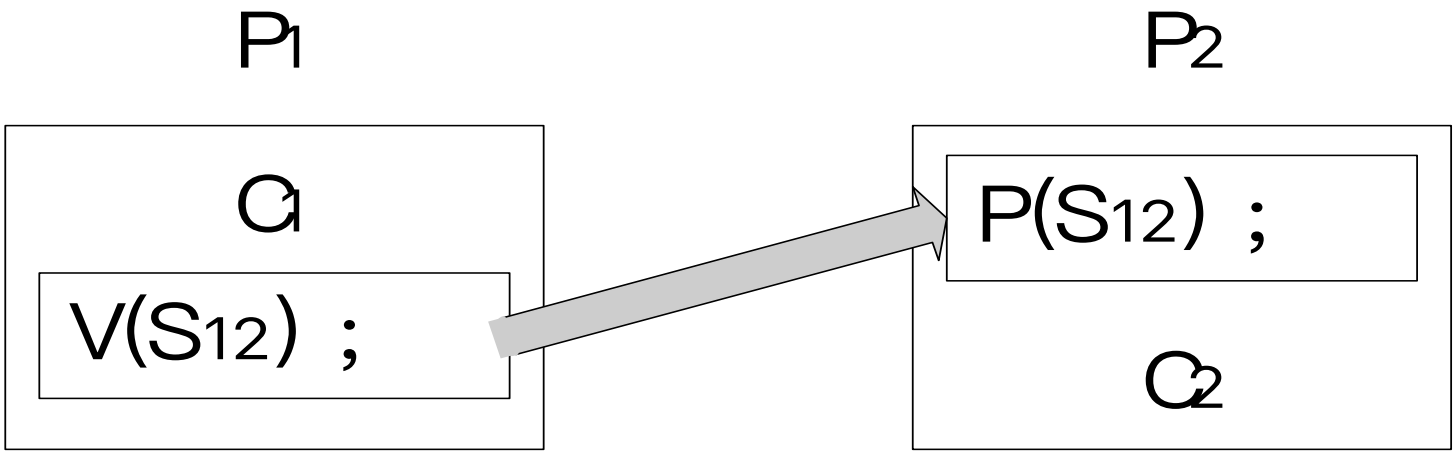
互斥在一起：互斥的信号量的PV操作在同一进程中





（同步）Synchronization using semaphore

- Consider two concurrently running processes P1 and P2, P1 with a statement C1 and P2 with a statement C2. suppose we require that C2 be executed only after C1 has completed
- We can define a common semaphore S12, initialized to 0.



同步分两边：同步的信号量的PV操作在不同进程中





10. 信号量与PV原语

信号量（Semaphore）表示系统中资源实体数目或资源使用情况的整型量，其值只能由PV原语操作改变。

n个进程共享m个资源，信号量变化范围

$$(m-n) \leqslant S \leqslant m$$

P(S)：代表申请使用资源的操作

S＝S-1；

若S＜0，则该进程被阻塞后与该信号相对应的对列中，然后转进程调度；

若S≥0 ，则，调用P(S)原语的进程继续运行。

V(S)：代表释放归还资源的操作

S＝S+1；

若S≤0，则唤醒一个等待S的进程后，调用P(S)原语的进程继续运行或转进程调度；

若S＞0，则，调用V(S)原语的进程继续运行。





The structure of Producer/Consumer process

Producer	Consumer
P(empty); P(mutex); //进入区	P(full); P(mutex); //进入区
one unit --> buffer;	one unit <-- buffer;
V(mutex); V(full); //退出区	V(mutex); V(empty); //退出区

生产者消费者问题：

一个互斥：对buffer的互斥访问

两个同步：有时必须先生产再消费；

有时必须先消费再生产





Readers-Writers Problem

Writer

```
P(Wmutex);
   write;
V(Wmutex);
```

Reader

```
P(Rmutex);
      if (Rcount == 0)
            P(Wmutex);
  ++Rcount;
V(Rmutex);
  read;
P(Rmutex);
  --Rcount;
      if (Rcount == 0)
            V(Wmutex);
V(Rmutex);
```

读者写者问题（没有同步、只有互斥）：

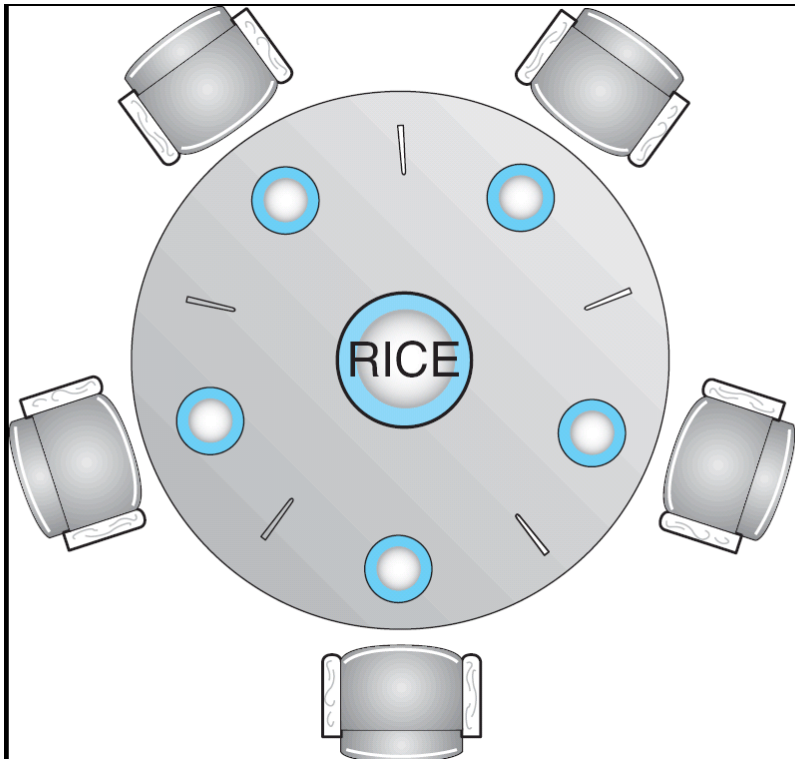
写者：一个互斥：对内容的互斥访问

读者：两个互斥：对内容、Rcount的互斥访问





Dining-Philosophers Problem



哲学家就餐问题：

多个变量的共同互斥访问





Dining-Philosophers Problem (Cont.)

- Shared data
  - Bowl of rice (data set)
  - Semaphore chopstick [5] initialized to 1

- The structure of Philosopher *i*:

```
While (true)  {
        wait ( chopstick[i] && ( chopStick[ (i + 1) % 5] );

                //  eat

        signal ( chopstick[i]  && chopstick[ (i + 1) % 5] );
                //  think
}
```





Monitors

A programming language construct

A collection of procedures, variables, data structures

Access to it is guaranteed to be exclusive.

- By the compiler (not programmer)

Monitors use separate mechanisms for the two types of synchronization

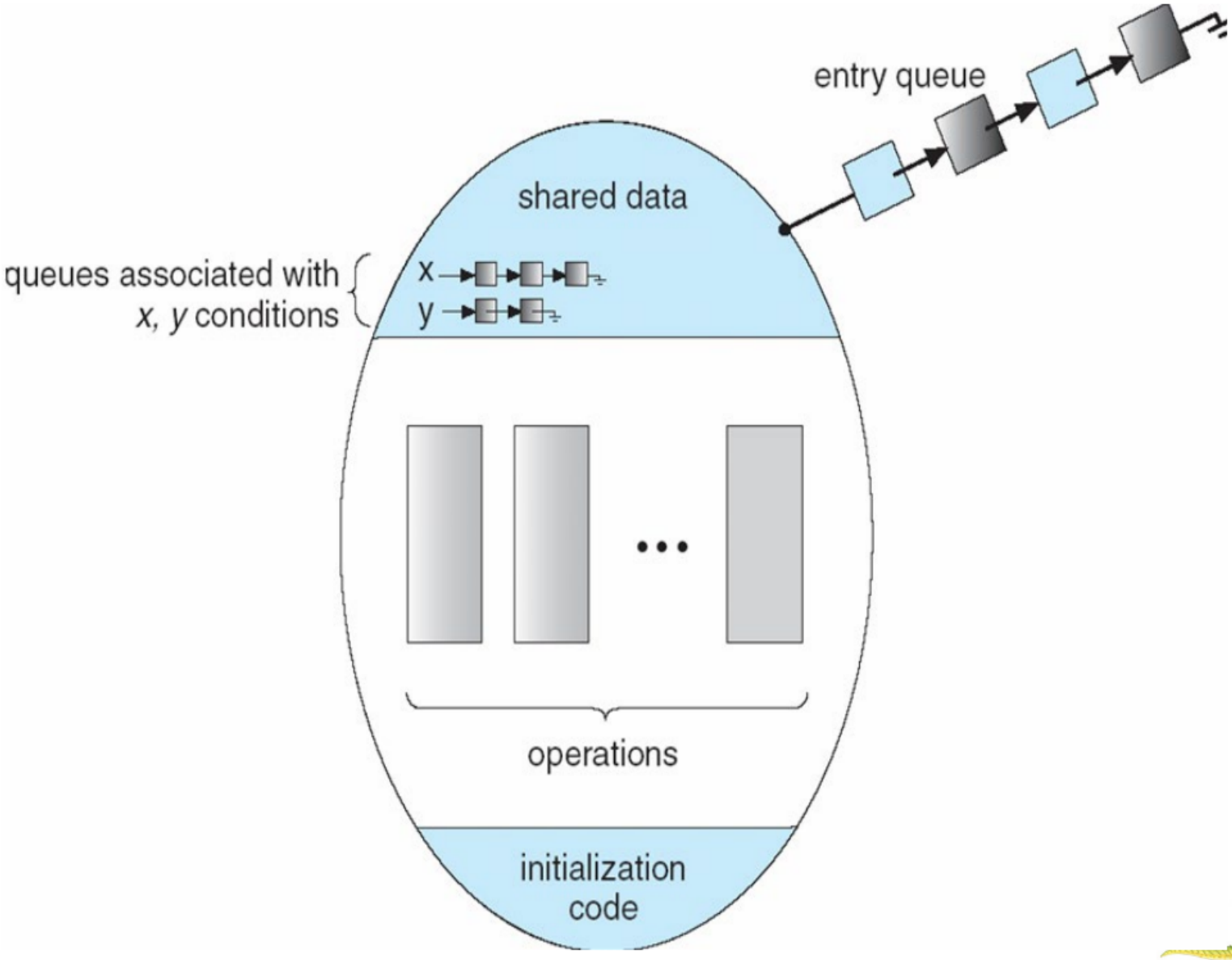
- use locks for mutual exclusion
- use condition variables for ordering constraints

monitor = a lock + the condition variables associated with that lock





Monitor with Condition Variables







例题

【单选】有m个进程共享同一临界资源，若使用信号量机制实现对一临界资源的互斥访问，则信号量的变化范围是（　）。

A.1至 -(m-1)　B.1至m-1

C.1至-m　　D.1至m

答案：　A





例题

【单选】当一进程因在记录型信号量S上执行P(S)操作而被阻塞后，S的值为（B）。

A.>0　　B.<0　　C.≥0　　D.≤0

答案：　B





例题

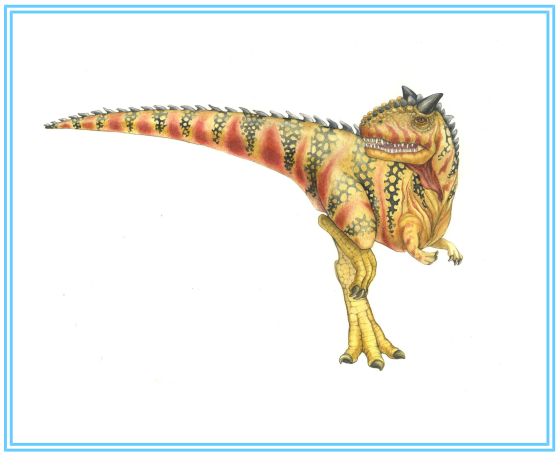
【单选】当一进程因在记录型信号量S上执行V(S)操作而导致唤醒另一进程后，S的值为（ ）

A.>0　　B.<0　　C.≥0　　D.≤0

答案： D



第7章 死锁



Silberschatz, Galvin and Gagne ©2009



第7章 死锁 概要总结

- The Deadlock Problem
- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

为什么：死锁为什么产生

是什么：死锁的特点?

四大必要条件：互斥、占有并等待、非抢占、循环等待

怎么办：如何解决死锁?

死锁预防：破坏四个必要条件

死锁避免：

单实例：检测环

多实例：银行家算法

死锁检测：

单实例：检测环

多实例：死锁检测算法

死锁恢复：牺牲某个/多个进程





The Deadlock Problem

A set of blocked processes each holding a resource and waiting to acquire a resource held by another process in the set.

The resources may be either physical resources or logical resources.





Necessary Conditions for Deadlock

Mutual exclusion: only one process at a time can use a resource

Hold and wait: a process holding at least one resource is waiting to acquire additional resources held by other processes

No preemption: a resource can be released only voluntarily by the process holding it, after that process has completed its task

Circular wait: there exists a set $\{P_0, P_1, \ldots, P_0\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1, P_1 is waiting for a resource that is held by

$P_2, \ldots, P_{n-1}$ is waiting for a resource that is held by P_n, and P_n is waiting for a resource that is held by P_0.





Chapter 7: Deadlocks(死锁)

怎么办：如何解决死锁?

死锁预防：破坏四个必要条件

互斥：难以破坏

持有并等待：执行前申请所有资源/只有不持有资源才能申请

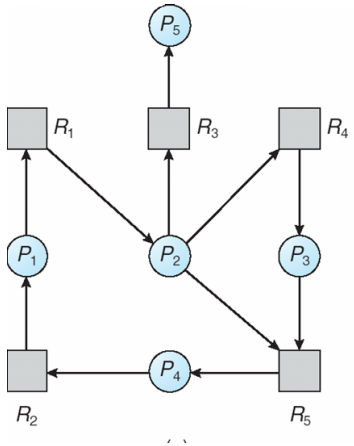
非抢占：CPU/内存可抢占，I/O有时不好抢占

循环等待：资源静态分配

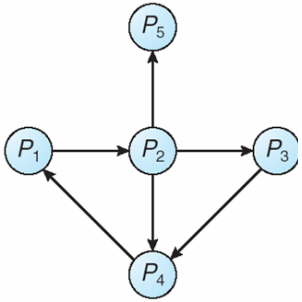
死锁避免：

单实例：检测环（资源分配图）

多实例：银行家算法



资源分配图



等待图

死锁检测：

单实例：检测环（等待图）

多实例：死锁检测算法

死锁恢复：牺牲某个/多个进程





死锁检测与银行家算法对比

死锁检测算法：

1. (a) *Work* = *Available*

   (b)For *i* = 1,2, …, *n*, if *Allocationi* ≠ 0, then

   Finish[i] = false;otherwise,

   Finish[i] = *true*

2. Find an index *i* such that both:

   (a) *Finish*[*i*] == *false*

   (b) *Requesti* ≤ *Work*

   If no such *i* exists, go to step 4

银行家算法：

1. (a) *Work* = *Available*

   (b)Finish [*i*] = *false* for *i* = 0, 1, …, *n*- 1

2. Find an *i* such that both:

   (a) *Finish* [*i*] == *false*

   (b) *Needi* ≤ *Work*

   If no such *i* exists, go to step 4





死锁检测与银行家算法对比

死锁检测算法：

3. *Work* = *Work* + *Allocation*$_i$
   Finish[*i*] = *true*
   go to step 2

4. If *Finish*[*i*] == false, for some *i*, 1 ≤ *i* ≤ *n*, then the system is in deadlock state. Moreover, if *Finish*[*i*] == *false*, then P_i is deadlocked

银行家算法：

3. *Work* = *Work* + *Allocationi*
   Finish[*i*] = *true*
   go to step 2

4. If *Finish* [*i*] == true for all *i*, then the system is in a safe state





例题

某系统采用了银行家算法，则下列叙述正确的是（B）。

A.系统处于不安全状态时一定会发生死锁

B.系统处于不安全状态时可能会发生死锁

C.系统处于安全状态时可能会发生死锁

D.系统处于安全状态时一定会发生死锁

答案：B





例题

【简答】什么是死锁？产生死锁的原因和必要条件是什么？

n(1)　一组进程中每个进程无限期地等待另一进程所占用且永远不会释放的资源，此时系统处于死锁

n(2)　原因：a.系统提供的资源有限；b.进程推进顺序不当

n（3）必要条件：互斥、非抢占、占有并等待、循环等待



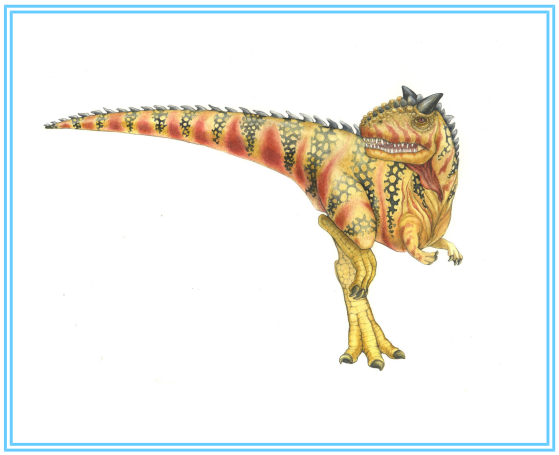


例题

- 在解决死锁问题的几个方法中，哪种方法最容易实现？哪种方法使资源的利用率最高？
- (1) 死锁预防方法，主要是破坏产生死锁的必要条件。该方法是最容易实现的，系统资源利用率较低
- (2) 死锁避免方法，例如银行家算法，需要较多的数据结构，实现起来比较困难，但资源利用率最高。
- （3）死锁检测方法是基于死锁定理设计的，定期运行该算法对系统的状态进行检测，发现死锁便予以解除。其中，需要比较一下各种死锁解除方案的



End of Exercise Lesson



Silberschatz, Galvin and Gagne ©2009